

Building Cooperative Robot Behavior from Basic Action Modules

An LLM-driven control framework for two cooperating AR4 arms

Jan M. Straub

Supervisor: Prof. Dr. Artur Andrzejak

July 20, 2026

Institute for Computer Science — Artificial Intelligence for Programming
Ruprecht-Karls-University of Heidelberg

1 The problem

The reasoning–execution gap, and why cooperation is the hard case

2 Pillar 1 — Grounding

A 20-operation vocabulary makes a small, local VLM reliable

3 Pillar 2 — Cooperation emerges

Handoffs and parallelism composed by the model

4 Pillar 3 — The boundary

The one failing task, what the ablations really show, limitations

5 Conclusion

The five contributions, the path to hardware, future work

Condensed Results

Thesis in one sentence

A **locally hosted vision-language model (VLM)**, grounded in a structured pool of **20 basic operations**, turns natural language into reliable **robot behavior**, with **no fine-tuning and no hard-coded sequencing**.

Solved

1.000 success rate on all single-robot tasks and dual-arm handovers.

Emergent

Cooperation is reasoned, not scripted. Negotiation lifts the default model's dual-robot success.

One gap

Concurrent bimanual lift fails: target selection is not partner-aware, plus a simulator attachment limit.

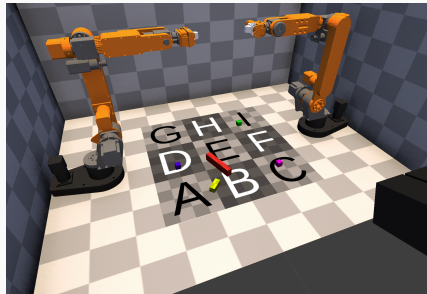
The Problem

The Reasoning–Execution Gap

- Classical robots: precise, but caged. Rigid paths, no tolerance for the unexpected.
- VLMs bring world knowledge and reasoning, but cannot reliably turn abstract reasoning into *physical* action.
- Models often ignore **past physical state** when planning the next move.

Why cooperation is the hard case

Two arms must **avoid collision**, **share a workspace**, and **synchronize timing**.



Two AR4 [1] arms, a shared workspace, and a stereo camera in Unity [2].

Two Design Commitments

Run the model *locally*

- No cloud cost, no external dependency.
- No sensitive scene data leaving the machine.
- A compact VLM is enough; no frontier model required.

Ground the model in *modular operations*

- Reasoning restricted to a vocabulary of *executable* actions.
- Separates semantic understanding from physical execution, giving a predictable control rate.
- Vision resolves targets live; no task-specific training.

Default backend model: **Magistral Small 2509 (24B, 4-bit) [3]**

Background

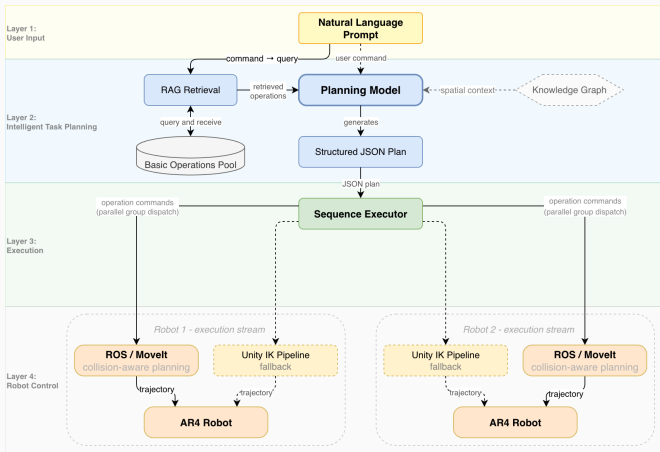
- **End-to-end VLA** (RT-2, PaLM-E) [4, 5]: great generalization, but data-hungry and opaque.
- **Code-as-Policies** [6]: compositional and transparent, but tied to its API set.
- **Framework-scale** (ROS-LLM, AutoRT) [7, 8]: practical scaffolding, but no joint reasoning over a *shared* object.
- **RoCo** [9]: per-robot agents negotiate, but free-form way-points.

Research gap

Two arms reasoning *jointly* about a single shared object, cooperation that *emerges* from reasoning rather than being engineered from outside, is comparatively unexplored.

Pillar 1 — Grounding

System Architecture



Built on Unity, ROS 2 [10], and MoveIt [11].

The Operation Pool

Operations organized by **complexity**, four tiers:

Atomic (8) indivisible, no perception: gripper, signal, wait.

Basic (6) single dispatch, introduces perception.

Intermediate (4) composite single-robot: placement, stereo detection, scene analysis.

Complex (2) full pipelines: grasp planning, handoff reception.

Why it grounds: model can only express plans the robot can actually run.

Operation Registry Map (20 operations)

Operation complexity						
	perception	navigation	manipulation	state_check	coordination	sync
			1. grasp_object		1. receive_handoff	
	1. analyze_scene 2. detect_object_stereo		1. place_between_objects 2. place_object			
basic	1. detect_all_fields 2. detect_field 3. generate_point_cloud	1. adjust_end_effector_orientation 2. move_to_coordinate 3. return_to_start_position				
atomic			1. control_gripper 2. release_object	1. check_robot_status	1. yield_workspace	1. reset_simulation 2. signal 3. wait 4. wait_for_signal

Two more Grounding Components

Reflection recovery

- On failure, re-query the model with a structured *hint* and recovery suggestions (Reflexion-style verbal feedback [12]).
- Closes the perception–action loop at the language level.
- Eligibility is gated: no point re-querying a sync barrier.

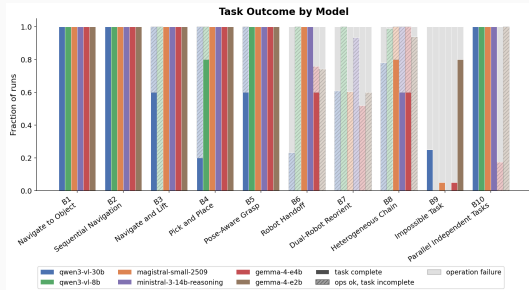
Knowledge graph

- Robots / objects / regions as nodes; reachability, proximity, grasps as edges.
- Answers “who can reach X?”, “viable handoff?” via multi-hop queries.
- **Spoiler:** measured benefit is *not* positive (Pillar 3).

Grounding $\Rightarrow$ Reliability

- **B1–B5** (navigate $\rightarrow$ grasp $\rightarrow$ full pick-and-place): **success over all 25 runs**, zero hallucinated ops, zero retries.
- *Identical* operation sequence on all 5 runs of each benchmark.
- **B8** (long chain): 20 sub-tasks, **85 operations/run**, 5 cycles, all executed, no cascading errors.

Cost is dominated by *contact-rich* primitives (grasp_object, place_object).



Magistral, Ministral, and both Gemma 4 models clear the full single-robot gradient; the Qwen3-VL models fail from B3 onward by truncating plans.

Pillar 2 — Cooperation Emerges

The Model Composes

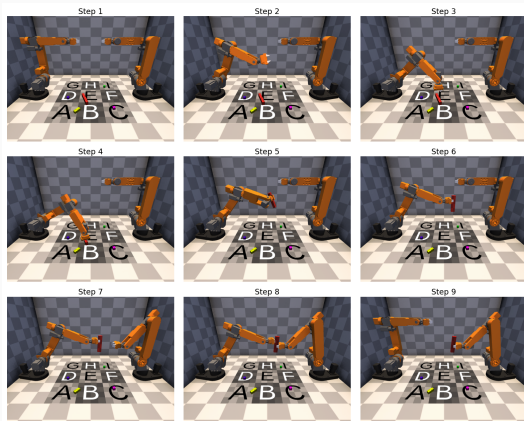
Two general primitives

- `signal / wait_for_signal`: one robot pauses at a checkpoint until the other signals readiness.
- **Parallel groups**: operations dispatched concurrently when no ordering is needed.

The executor imposes **no** coordination policy. The model decides *where* synchronization is needed and *which* steps can run in parallel, guided only by general rules in the prompt, never an explicit script.

The Handoff

- Magistral and Ministral: 5/5 runs; all operations succeed first try.
- The plan places a **synchronization barrier** so Robot2 receives only after Robot1 signals.
- Gemma-4B partial (3/5); the Qwen3 models and Gemma-2B break down at plan construction.



Handoff sequence.

Emergent Parallelism

- 4/6 models: 40/40 operations, zero retries.
- Applying the prompt's independent-parallel rule.
- Heavy manipulation phases run concurrently; detection kept sequential, a conservative, stable choice.

Takeaway

The model expresses physical simultaneity when the task allows it, the same capability that matters for the unsolved B7.

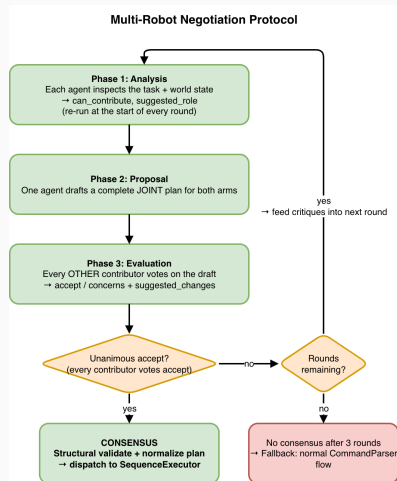
Negotiation

A single model planning both arms under-specifies roles;
implicit conflicts surface only at execution.

Protocol:

1. **Analyze**: each agent assesses its own role (parallel).
2. **Propose**: one agent/round drafts a *complete* plan (round-robin).
3. **Evaluate**: peers accept/reject with justification; **unanimity** required. Rejections feed the next proposal.

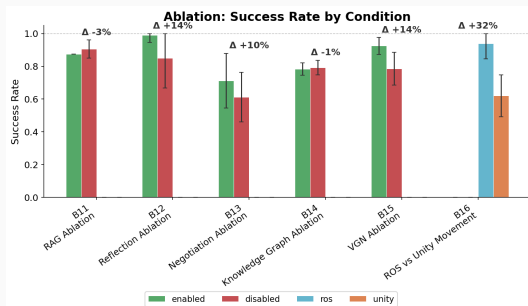
No consensus in 3 rounds → single-model parsing.



Negotiation Pays Off

	Task SR	Ops/run
Negotiation on	0.713 (57/80)	35.8
Negotiation off	0.613 (49/80)	29.9

- **+10 pp** task success for the **default model**.
- Roughly neutral for three backends; *negative* for two (−19, −21 pp).
- Pays off when the single-shot plan is already close to correct; doesn't rescue weaker planners from role conflicts.



Per-component ablation deltas (Magistral; $n = 20$ for B11–B13, B16; $n = 5$ for B14–B15).

Pillar 3 — The Boundary

Concurrent Bimanual Lift

All six models fail. Three failure modes:

1. Signal primitives never dispatched once control crosses robots outside a parallel group.
2. Final lift: two *independently* chosen targets land too close together, violating the separation check.
3. Deeper sim limit: a Unity Transform has one parent, so a single object can't attach to two grippers at once.

The gap

Partner-aware target selection. Each arm picks a valid target; nothing checks one *against the other*.

Ablation Synthesis

Component	Effect	Reading
ROS/Movelt	+32 pp	Unity-only grasp timeouts; plus collision-aware planning
Reflection	+14 pp (task)	Clears boundary-reach failures; +7 pp op; doubles time when it fires
Negotiation	+10 pp	Default model only; neutral/negative on the other five
RAG [13]	net cost	Helps no backend; hurts 3 of 6 (−7 pp pooled)
Knowledge graph	small <i>cost</i>	Prompt over-anchoring, not a broken component
VGN grasp	+35 pp Magi (+33 pp pooled)	center-of-object grasp, not predicted orientation

Caveat

Point estimates, one simulated environment: $n = 20$ per condition for RAG, Reflection, Negotiation and ROS, $n = 5$ for the knowledge graph and ROS. Enough to expose deterministic failure modes, not to resolve effects within one standard deviation.

AutoRT Adaptation & Safety Gate

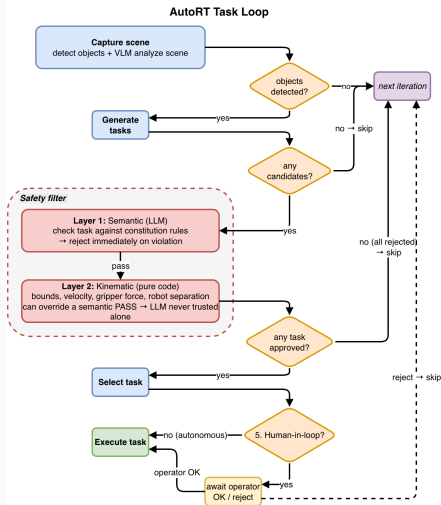
Re-targeted AutoRT for *fixed-base* manipulation.

Two-layer safety (Robot Constitution):

- Semantic VLM check
- Hard kinematic check the semantic layer can't override.

Result: never admits an unsafe task.

Slightly over-rejects aggressive-but-safe phrasing.



Limitations

Sensing

Stereo depth degrades on textureless surfaces, so VGN's predicted poses are overridden by the top-down heuristic.

Latency

VLM parse ≈ 20 s median, which prevents *reactive* control; re-plan only between motions, never during.

Sim-to-real

Robot joint values all tuned for simulation, so they need revalidation on hardware.

Validation

5/6 models forward impossible inputs; the guard must be pre-parse.

Conclusion

Five Contributions

1. **Retrieval-grounded operation pipeline:** a hierarchical 20-operation registry retrieved on demand via RAG, with two-phase decomposition and runtime variable-passing. No hard-coded sequences, no fine-tuning.
2. **Runtime-composed coordination:** handoff, parallelism, and waiting built from general `signal/wait` and `parallel-group` primitives, not scripted into the executor.
3. **Consensus-gated negotiation:** per-robot agents analyze/propose/evaluate under unanimity; +10 pp on role-ambiguous tasks for the default model.
4. **Operation-gated reflection:** retry with functional recovery hints on eligible failures; +14 pp task success for the default model.
5. **AutoRT adaptation:** fixed-base task generation plus a two-layer Robot Constitution with a hard kinematic layer the semantic check cannot override.

Path to a Real Robot

- Hardware abstraction boundary: `-env real` swaps camera and hardware adapters.
- Switch control to **ROS**: ROS/MoveIt collision-aware planning, the correct route for physical AR4 hardware, already dual-robot configured.

Two remaining blockers

Depth sensing: a LIDAR source unlocks the VGN pipeline.

Force-torque feedback: a wrist sensor replaces the contact-duration heuristic.

- **Depth upgrade:** a LIDAR sensor fully unlocks VGN-grade grasping.
- **Persistency:** a persistent volumetric map using SLAM
- **Bimanual pre-pass:** detect simultaneous grasp language, route to a bimanual-aware prompt to solve bimanual gap.
- **Pre-parse validation:** robot-ID and object-existence checks before any model call.
- **Faster inference:** distilled task-specific models and/or speculative parsing to shrink the 20 s window.

Summary

Summary

What works

- Single-robot: works in 25/25 runs.
- Long chain: 85 ops/run, 5 cycles, no cascading errors.
- Handoff: works and correctly realized sync barrier.
- Parallelism: works for 4/6 backends.
- Safety gate: no false-accept, reproducible.

What each component contributes

- VGN grasp: +35 pp Magistral, +33 pp pooled.
- ROS/Movelt: +32 pp, Unity-only grasp timeouts.
- Reflection: +14 pp task.
- Negotiation: +10 pp, default model only.
- RAG: model-dependent; KG (B14): small cost.

The one open gap

- Concurrent bimanual B7: all six models fail.
- Cause: partner-aware target selection.

Demo & Questions

`github.com/JanMStraub/Cooperative-Robot-Behavior-from-Basic-Action-Modules`

References

- [1] C. Annin, "Annin robotics," Accessed: 2026-06-09, 2026. [Online]. Available: <https://anninrobotics.com>
- [2] Unity Technologies, "Unity," 2026, game development platform. [Online]. Available: <https://unity.com/>
- [3] Mistral AI, "Magistral-small-2509," <https://huggingface.co/mistralai/Magistral-Small-2509>, 2025, large Language Model.
- [4] A. Brohan, N. Brown, J. Carbajal et al., "Rt-2: Vision-language-action models transfer web knowledge to robotic control," 2023. [Online]. Available: <https://arxiv.org/abs/2307.15818>
- [5] D. Driess, F. Xia, M. S. M. Sajjadi et al., "Palm-e: An embodied multimodal language model," 2023. [Online]. Available: <https://arxiv.org/abs/2303.03378>
- [6] J. Liang, W. Huang, F. Xia et al., "Code as policies: Language model programs for embodied control," 2023. [Online]. Available: <https://arxiv.org/abs/2209.07753>
- [7] C. E. Mower, Y. Wan, H. Yu et al., "A robot operating system framework for using large language models in embodied ai," *Nature Machine Intelligence*, vol. 8, no. 3, p. 313–325, Mar. 2026. [Online]. Available: <http://dx.doi.org/10.1038/s42256-026-01186-z>
- [8] M. Ahn, D. Dwibedi, C. Finn et al., "Autort: Embodied foundation models for large scale orchestration of robotic agents," 2024. [Online]. Available: <https://arxiv.org/abs/2401.12963>
- [9] Z. Mandi, S. Jain, and S. Song, "Roco: Dialectic multi-robot collaboration with large language models," 2023. [Online]. Available: <https://arxiv.org/abs/2307.04738>
- [10] S. Macenski, T. Foote, B. Gerkey et al., "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, May 2022. [Online]. Available: <http://dx.doi.org/10.1126/scirobotics.abm6074>
- [11] D. Coleman, I. Sukan, S. Chitta et al., "Reducing the barrier to entry of complex robotic software: a moveit! case study," 2014. [Online]. Available: <https://arxiv.org/abs/1404.3785>
- [12] N. Shinn, F. Cassano, E. Berman et al., "Reflexion: Language agents with verbal reinforcement learning," 2023. [Online]. Available: <https://arxiv.org/abs/2303.11366>
- [13] P. Lewis, E. Perez, A. Piktus et al., "Retrieval-augmented generation for knowledge-intensive nlp tasks," 2021. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [14] M. Breyer, J. J. Chung, L. Ott et al., "Volumetric grasping network: Real-time 6 dof grasp detection in clutter," 2021. [Online]. Available: <https://arxiv.org/abs/2101.01132>

Additional Slides

Backup — Bimanual Lift per-run Outcomes

Run	Ops ok	Dur (s)	Failing op
1	4/8	111	wait_for_signal
2	4/10	43	wait_for_signal
3	6/7	211	move_to_coordinate (collision)
4	6/7	206	move_to_coordinate (collision)
5	4/10	46	wait_for_signal

Two clusters: mis-realized barrier, never executed (1,2,5), and lift-target collision at ~ 0.10 m (3,4).

Backup — What VGN [14] changes

Held & lifted	VGN on	VGN off
Magistral	14/20 (0.700)	7/20 (0.350)
Pooled, 6 models	0.750	0.417

- Every backend improves: +20 pp (Ministral) to +65 pp (Gemma-4B).
- Success = held and lifted clear of the table.

Mechanism: position, not orientation

- **Same orientation both conditions:** top-down, yaw from the object's long axis. VGN's own poses fail motion planning and are discarded.
- **The difference is position:** VGN grasps the detected object *center*; the baseline's contact point sits off-center on angled cubes and slips.

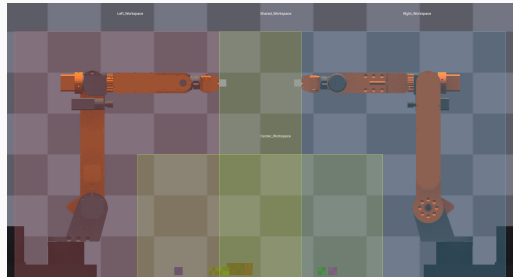
Scope

Position comes from the WorldState oracle, so B15 isolates grasp *positioning*. Recovering VGN's learned pose needs a real depth sensor.

Backup — Collision Safety Layers

1. **Static pre-check** (Python, before dispatch):
CoordinationVerifier checks workspace bounds, object conflicts, deadlock detection on the planned targets.
2. **Workspace regions**: left / right / shared zones;
a contested zone is handed over explicitly via `yield_workspace`.
3. **Runtime guard** (Unity, every physics step):
ProximityGuard freezes motion at 0.25 m end-effector distance, resumes at 0.35 m; link-to-link limit 0.15 m.

The LLM is never trusted for safety: layers 1 and 3 are code, not reasoning.



Backup — Single-Robot Operation Latencies

Operation	Bench	n	Mean (s)
detect_object_stereo	B1–B5	30	0.80
move_to_coordinate	B1–B3	20	4.37
grasp_object (B3)	B3	5	19.84
grasp_object (B4)	B4	5	10.93
place_object	B4	5	34.43

Perception is cheap and stable; contact-rich manipulation dominates and varies with object pose.

Backup — Model × Task Matrix

